Reporte Técnico de Actividades Práctico-Experimentales Nro. 004

Análisis comparativo de herramientas de simulación mediante un modelo común

1. Datos de identificación del estudiante y la práctica

Nombre del estudiante(s)	Emilio Jose Galvez Merchan
Asignatura	Simulación
Ciclo	5 A
Unidad	U1: Introducción a la Simulación
Resultado de aprendizaje de la unidad	R1. Aplica análisis estadísticos de los datos de la Simulación para mejorar la calidad de los resultados, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	004
Tipo	Grupal
Título de la práctica	Análisis comparativo de herramientas de simulación mediante un modelo común
Nombre del docente	Ing. José O. Guamán Q.
Fecha	22 y 23 de octubre de 2026
Horario	Jueves 10h30 a 11h30 · Viernes 07h30 a 09h30
Lugar	Aula
Tiempo planificado en el sílabo	3 horas

Tabla 1: Datos de identificación del estudiante y de la práctica.

2. Objetivos de la práctica

Comparar diferentes lenguajes y plataformas para la implementación de modelos de simulación.

- Implementar un modelo sencillo en diferentes herramientas.
- Comparar sintaxis, facilidad de implementación y visualización.
- Identificar el ámbito de aplicación de cada herramienta.

3. Materiales, reactivos, equipos y herramientas

- Computador.
- Sistema operativo Linux o macOS.
- Python, con las bibliotecas NumPy y Matplotlib.
- SimPy.
- Java / JDK.
- NetLogo.

4. Procedimiento / Metodología ejecutada

- Se analizó la práctica APE 4 de forma grupal, para buscar el problema a resolver o el modelo que debemos aplicar, cómo y con qué herramientas.
- Se identificaron las principales variables del modelo matemático.
- Una vez identificado el problema, lo que debíamos hacer y lo que la práctica requiere para cumplirse, hicimos uso de un modelo de inteligencia artificial. Para ello, dimos instrucciones paso a paso de lo que queríamos tener como resultado, evaluando lo que nos entregaban, y cuando ocurrieron errores (que sí existieron), los fuimos corrigiendo en conjunto con el modelo.
- Evaluamos los resultados y nos aseguramos de que se genere correctamente el documento .csv que requiere la actividad general. Hasta aquí llegó el alcance grupal.
- Luego se hizo una búsqueda de fuentes bibliográficas para responder las preguntas de control. Busqué las más adecuadas con palabras clave relacionadas con el tema y posteriormente extraje los PDFs de estas fuentes.
- Se adjuntó cada PDF a Mendeley para extraer los .bib y a un cuaderno de NotebookLM para poder dar respuesta a ciertas preguntas de control.
- Nuevamente se hizo uso de un modelo de inteligencia artificial, esta vez como guía para utilizar NetLogo e implementar la simulación y la parte visual que muestre los resultados del .csv.
- Con todo listo, se procedió a hacer el informe hasta finalizarlo.

5. Resultados esperados

5.1 Un modelo SIR implementado en NetLogo

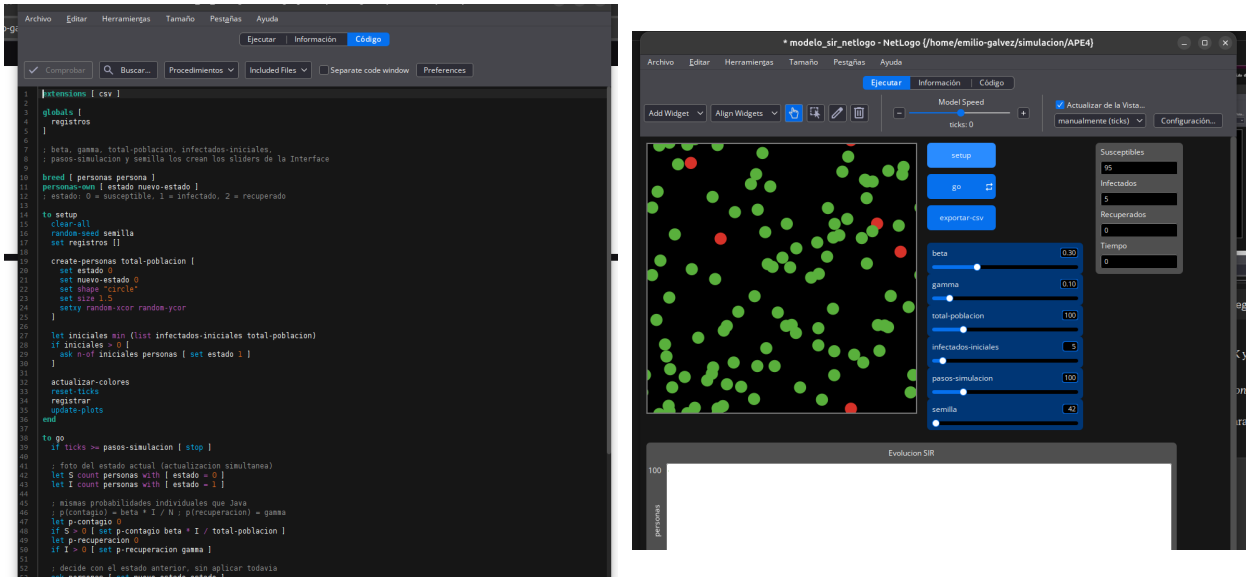
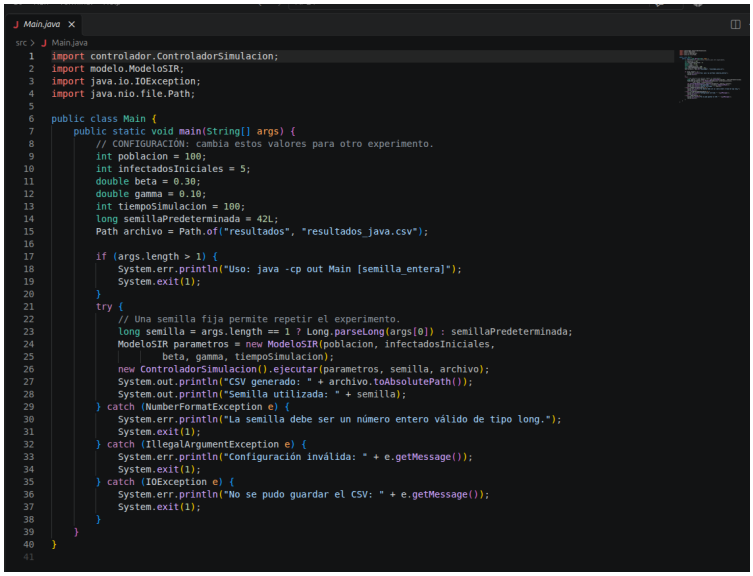


Figura 1: Modelo SIR en NetLogo: pestaña de código (izquierda) e interfaz con los botones, sliders, monitores y gráfica (derecha).

5.2 Modelo equivalente implementado en Java



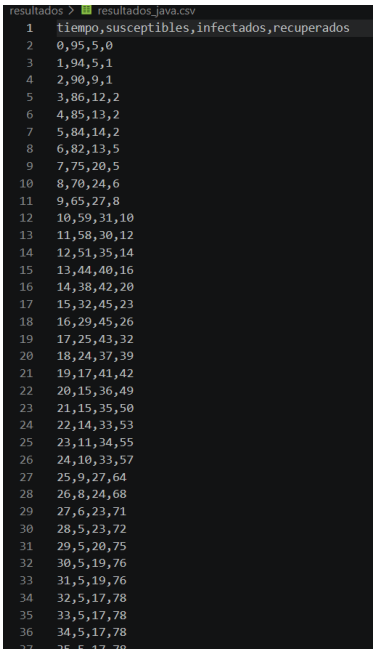
```java
import controlador.ControladorSimulacion;
import modelo.ModeloSIR;
import java.io.IOException;
import java.nio.file.Path;

public class Main {
    public static void main(String[] args) {
        // CONFIGURACIÓN: cambia estos valores para otro experimento.
        int poblacion = 100;
        int infectadosIniciales = 5;
        double beta = 0.30;
        double gamma = 0.10;
        int tiempoSimulacion = 100;
        long semillaPredeterminada = 42L;
        Path archivo = Path.of("resultados", "resultados_java.csv");

        if (args.length > 1) {
            System.err.println("Uso: java -cp out Main [semilla_entera]");
            System.exit(1);
        }
        try {
            // Una semilla fija permite repetir el experimento.
            long semilla = args.length == 1 ? Long.parseLong(args[0]) : semillaPredeterminada;
            ModeloSIR parametros = new ModeloSIR(poblacion, infectadosIniciales,
                    beta, gamma, tiempoSimulacion);
            new ControladorSimulacion().ejecutar(parametros, semilla, archivo);
            System.out.println("CSV generado: " + archivo.toAbsolutePath());
            System.out.println("Semilla utilizada: " + semilla);
        } catch (NumberFormatException e) {
            System.err.println("La semilla debe ser un número entero válido de tipo long.");
            System.exit(1);
        } catch (IllegalArgumentException e) {
            System.err.println("Configuración inválida: " + e.getMessage());
            System.exit(1);
        } catch (IOException e) {
            System.err.println("No se pudo guardar el CSV: " + e.getMessage());
            System.exit(1);
        }
    }
}
```

Figura 2: Clase *Main* del programa en Java, donde se configuran los parámetros del experimento.

5.3 Archivo CSV generado por Java



```
tiempo,susceptibles,infectados,recuperados
0,95,5,0
1,94,5,1
2,90,9,1
3,86,12,2
4,85,13,2
5,84,14,2
6,82,13,5
7,75,20,5
8,70,24,6
9,65,27,8
10,59,31,10
11,58,30,12
12,51,35,14
13,44,40,16
14,38,42,20
15,32,45,23
16,29,45,26
17,25,43,32
18,24,37,39
19,17,41,42
20,15,36,49
21,15,35,50
22,14,33,53
23,11,34,55
24,10,33,57
25,9,27,64
26,8,24,68
27,6,23,71
28,5,23,72
29,5,20,75
30,5,19,76
31,5,19,76
32,5,17,78
33,5,17,78
34,5,17,78
```

Figura 3: Primeras filas del archivo *resultados_java.csv* (tiempo, susceptibles, infectados y recuperados).

5.4 Gráficas de evolución de susceptibles, infectados y recuperados

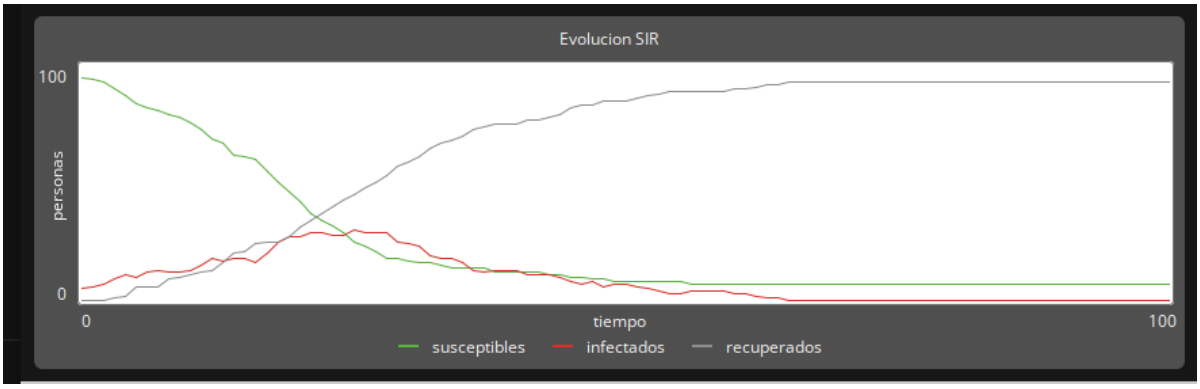


Figura 4: Evolución de susceptibles, infectados y recuperados durante los 100 pasos de la simulación.

5.5 Comparación de los resultados obtenidos

Tiempo	Java (S / I / R)	NetLogo (S / I / R)
0	95 / 5 / 0	95 / 5 / 0
2	90 / 9 / 1	90 / 9 / 1
15	32 / 45 / 23	32 / 45 / 23
16	29 / 45 / 26	29 / 45 / 26
57	2 / 0 / 98	2 / 0 / 98
100	2 / 0 / 98	2 / 0 / 98

Tabla 2: Comparación de resultados entre Java y NetLogo en seis tiempos de la simulación.

Comparé seis tiempos y en todos los valores de Java y NetLogo salen exactamente iguales, sin ninguna diferencia en S, I ni R. En las dos herramientas el pico de infectados fue de 45 (en los tiempos 15 y 16). En el tiempo 57 los infectados llegaron a 0, quedando 2 susceptibles y 98 recuperados, y todo se mantuvo así hasta el tiempo 100.

5.6 Visualización en NetLogo de los datos generados por Java

Para la visualización hice en NetLogo un lector del archivo *resultados/resultados_java.csv*. Con la extensión csv carga las 101 filas, se salta el encabezado y se queda con el tiempo, los susceptibles, los infectados y los recuperados. El botón **setup** carga los datos y dibuja las 100 personas en una cuadrícula de 10 × 10; **go** avanza un registro a la vez y **final** salta directo al último estado. En cada paso se pintan las personas según los conteos del CSV y se actualizan los monitores y la gráfica.

Así se ve cómo evoluciona la epidemia sin volver a simular nada: azul es susceptible, rojo es infectado y verde es recuperado. La Figura 5 muestra el estado final (2 susceptibles, 0 infectados y 98 recuperados) y la Figura 6 la interfaz completa con los botones, los monitores de S, I y R y la gráfica con las tres curvas. Ojo: la posición de las figuras es solo estética, no representa contactos reales entre personas.

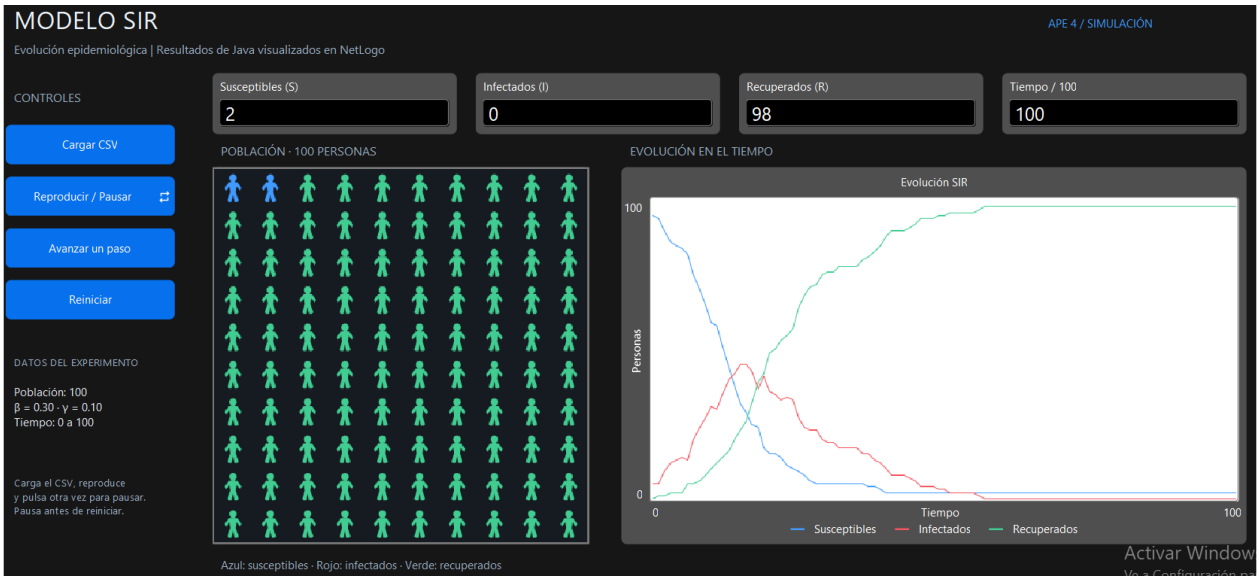


Figura 5: Estado final de la población en la visualización: 2 susceptibles (azul), 0 infectados y 98 recuperados (verde).

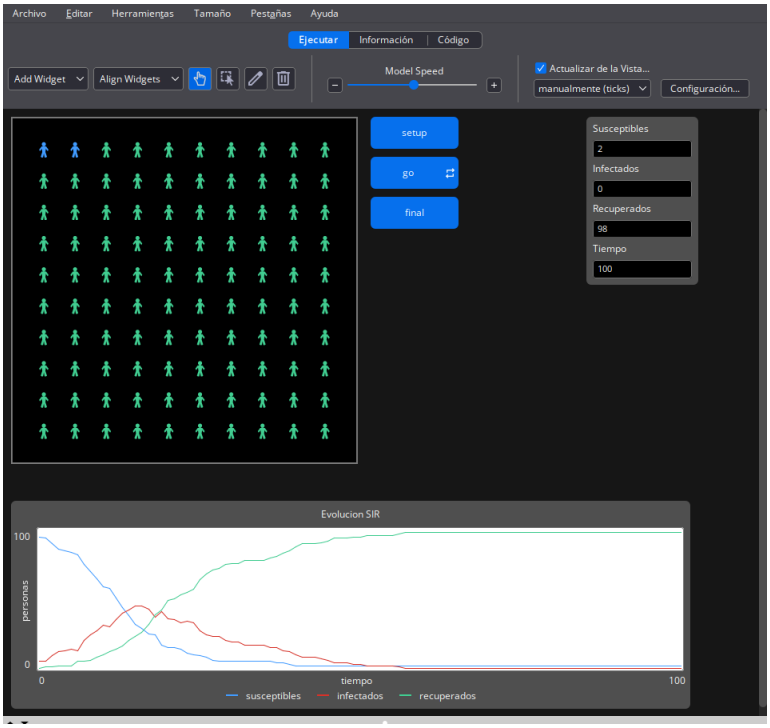


Figura 6: Interfaz del modelo lector de CSV en NetLogo, con los botones setup, go y final, los monitores de S, I y R y la gráfica de las tres curvas.

5.7 Ventajas y limitaciones de cada herramienta

Java. Me dio control total del modelo, de los parámetros y de los datos que se generan. Con la semilla 42 la corrida se puede repetir y el CSV guardó los 101 registros. Lo malo es que no trae interfaz visual, así que para ver la población o las curvas hace falta otra herramienta, y además toca programar a mano las transiciones, los conteos y las validaciones, que en NetLogo salen con menos código.

NetLogo. Es mucho más cómodo para mostrar y explicar el modelo: con las figuras, los colores, los monitores y la gráfica se entiende la evolución sin revisar el CSV fila por fila. Su límite aquí es que

depende de Java: si cambio beta, gamma o la población, primero tengo que volver a ejecutar Java. Además solo recibe conteos totales, así que no se puede saber quién contagió a quién.

El CSV como puente. Funcionó bien porque separa el cálculo de la visualización y deja reutilizar los mismos datos. Eso sí, no es comunicación en tiempo real y el archivo tiene que estar en la ruta correcta. Si se comparara con una simulación independiente hecha solo en NetLogo, las diferencias vendrían del generador aleatorio y del orden en que se usan los números; en este trabajo no afecta porque NetLogo solo reproduce lo que calculó Java.

5.8 Resumen: ventajas y limitaciones

Herramienta	Ventajas	Limitaciones
Java	• Las clases separan las reglas de la simulación de la exportación de resultados, y así es fácil modificar el modelo. • Se controlan a mano los parámetros y las transiciones, lo que ayuda a armar distintos experimentos. • Guarda todos los pasos, así que se pueden revisar los conteos y analizar la evolución con números.	• Las transiciones, los conteos y las validaciones se programan y revisan a mano: más trabajo. • No tiene interfaz gráfica; para mostrar la evolución visualmente se necesita otra herramienta.
NetLogo	• Los botones y monitores dejan explorar los resultados paso a paso y facilitan la presentación. • La gráfica junta S, I y R, y así se ve el pico de infectados y el estado final. • Las figuras y los colores ayudan a interpretar la población sin mirar cada registro.	• Aquí solo reproduce lo de Java; para cambiar los parámetros hay que volver a correr Java. • Los datos son conteos agregados: no se puede saber quién contagió a quién ni seguir trayectorias individuales.

Tabla 3: Ventajas y limitaciones de Java y NetLogo en la implementación realizada.

En resumen: Java se encargó del cálculo y NetLogo de mostrar e interpretar el resultado. Usarlos juntos cubrió las dos necesidades, aunque la visualización depende de los datos que Java genere antes.

6. Preguntas de control

1. ¿Qué representa el modelo SIR?

Representa cómo avanza una infección en una población dividida en susceptibles, infectados y recuperados, y deja ver cómo cambian esos grupos con el tiempo.

2. ¿Qué representan los estados S, I y R?

S son las personas que se pueden infectar, I las que ya están infectadas y R las que se recuperaron. En este modelo los recuperados no se vuelven a infectar.

3. ¿Qué función cumplen los parámetros β y γ?

β controla qué tan rápido se transmite la infección y γ qué tan rápido se recuperan. En nuestro programa cada susceptible se infecta con probabilidad β·I/N por paso y cada infectado se recupera con probabilidad γ. Con los datos iniciales eso da 1.5 % y 10 %, respectivamente.

4. ¿Por qué es importante utilizar los mismos parámetros en las tres herramientas?

Para comparar los resultados en las mismas condiciones; también hay que mantener las reglas de actualización y la duración de cada paso. En este trabajo Java generó los datos y NetLogo los visualizó, o sea que no se corrieron tres simulaciones independientes.

5. ¿Por qué Python, Java y NetLogo pueden producir resultados diferentes con el mismo modelo?

Porque cada uno puede usar generadores aleatorios, semillas u órdenes de ejecución distintos. En Java, *Random* repite su secuencia si se mantienen la semilla y las mismas llamadas [1], pero eso no asegura una secuencia idéntica en otro lenguaje. Por eso nuestro programa usa la semilla 42, para poder repetir el experimento.

6. ¿Qué ventajas ofrece NetLogo para representar sistemas basados en agentes?

Trae agentes, formas, colores y herramientas para observar sus estados [2]. En nuestra interfaz, las figuras y los contadores siguen los valores del CSV. Eso sí, la posición de cada figura es solo visual: no muestra contactos reales ni sigue a una persona en el tiempo.

7. ¿Qué ventajas ofrece SimPy para simulaciones basadas en eventos?

Deja organizar procesos, tiempos de espera y recursos compartidos dentro de un entorno de eventos, y sirve para representar clientes, servidores o vehículos que interactúan [3]. Esta respuesta es teórica: en el APE 4 trabajamos con Java y no se hizo una versión en SimPy.

8. ¿Qué ventajas ofrece Java para implementar modelos de simulación?

Permite organizar el programa en clases, guardar estados en arreglos y listas, controlar los parámetros y escribir archivos. En el proyecto separamos los datos del modelo, el controlador de la simulación y la exportación a CSV, sin necesitar bibliotecas externas.

9. ¿Qué función cumple el archivo CSV en la integración Python/Java → NetLogo?

Sirve para pasar los resultados de una herramienta a otra. En nuestro caso, cada fila trae tiempo, susceptibles, infectados y recuperados; NetLogo lo lee con *csv:from-file* [4], se salta el encabezado y usa los registros para actualizar las figuras, los monitores y las curvas.

10. ¿Qué ventajas tiene separar la generación de datos de la visualización?

Se puede revisar y reutilizar el mismo resultado sin simular otra vez. Además, se puede cambiar el diseño, pausar la reproducción o reiniciar las gráficas sin tocar las reglas del modelo, y es más fácil comparar lo que muestra NetLogo con lo que quedó guardado en el CSV.

7. Conclusiones

El modelo SIR se implementó en Java y se conectó con NetLogo mediante el archivo CSV. Los conteos de susceptibles, infectados y recuperados coincidieron en todos los tiempos comparados, así que la visualización refleja bien lo que calculó Java.

En Java tocó programar la simulación de forma explícita, con clases, tipos de datos y ciclos. En NetLogo, en cambio, los procedimientos y los elementos de la interfaz hicieron más fácil armar la parte visual y controlar la reproducción.

Con la práctica quedó claro que Java sirve para desarrollar la lógica y automatizar la generación de datos, y NetLogo para explorar y explicar modelos de población. Cuál usar depende de qué se quiera priorizar: el cálculo, la visualización o una mezcla de ambos.

8. Referencias bibliográficas

[1] Oracle, “Random”, *Java Platform Standard Edition 21 API Specification*. [En línea]. Disponible: docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/Random.html. Consulta: 24 de octubre de 2026.

[2] NetLogo, “Programming Guide”, *NetLogo 7.0.4 User Manual*. [En línea]. Disponible en la documentación oficial de NetLogo: docs.netlogo.org. Consulta: 24 de octubre de 2026.

[3] SimPy, “Overview”, *SimPy 4.1.2 Documentation*. [En línea]. Disponible: simpy.readthedocs.io/en/stable/index.html. Consulta: 24 de octubre de 2026.

[4] NetLogo, “CSV Extension”, *NetLogo 7.0.4 User Manual*. [En línea]. Disponible: docs.netlogo.org/csv.html. Consulta: 24 de octubre de 2026.

9. Anexos

Declaración de uso de IA

Usé el modelo **Big Pickle** como guía para aprender a manejar la herramienta de simulación NetLogo y para corregir errores en la lógica de la aplicación. También usé **Claude** (modelo Claude Sonnet 5.5, de Anthropic) como apoyo con la interfaz de NetLogo y para ordenar y redactar este informe. Además, usé NotebookLM para extraer información de las fuentes y responder algunas preguntas de control.

Prompt utilizado

> “Con el contexto del proyecto, ayúdame a elaborar una presentación de los resultados obtenidos.”

Prompt para NotebookLM

> “Revisa las fuentes que cargué en el cuaderno y respóndeme las siguientes preguntas basándote en ellas. Al final de cada respuesta, indícame el nombre del artículo o documento del que sacaste la información:
> - ¿Por qué Python, Java y NetLogo pueden dar resultados diferentes usando el mismo modelo?
> - ¿Qué ventajas tiene NetLogo para representar sistemas basados en agentes?
> - ¿Qué ventajas ofrece SimPy para desarrollar simulaciones basadas en eventos?”